

Chapter 14: Indexing

Q&A Examples

Question

What is the difference between **primary key** and **unique key** in databases?

Answer

- **Primary Key:** Ensures unique identification of each record and does not allow NULL.
- **Unique Key:** Also enforces uniqueness, but allows a single NULL value.

Question

Explain the difference between **Dense Index** and **Sparse Index**.

Answer

- **Dense Index:** Every search key value has an entry in the index. *Pro: Fast lookup, Con: Larger index size.*
- **Sparse Index:** Only some search key values are indexed. *Pro: Smaller index size, Con: Requires extra block access in search.*

Question

Suppose we build a dense index on a relation with 1,000,000 tuples. Each index entry is small, so assume 100 entries fit in a 4KB block.

- (a) How large will the index be if the relation has (i) 1,000,000 tuples, (ii) 100,000,000 tuples?
- (b) How many random block reads are required to perform a binary search on an index of 10,000 blocks?
- (c) Why is binary search on a large index costly, and how can a multilevel index

reduce the cost?

Answer

(a) Index size calculation:

- For 1,000,000 tuples:

$$\frac{1,000,000}{100} = 10,000 \text{ blocks } (\approx 40 \text{ MB}).$$

- For 100,000,000 tuples:

$$\frac{100,000,000}{100} = 1,000,000 \text{ blocks } (\approx 4 \text{ GB}).$$

Thus, with very large relations, the index itself can be several gigabytes.

(b) Binary search cost:

$$\lceil \log_2(10,000) \rceil = 14 \text{ random block reads.}$$

On a magnetic disk, if one random read takes 10 ms, total time = $14 \times 10 \text{ ms}$
= 140 ms.

$\Rightarrow$ Only ≈ 7 searches per second possible on a single disk.

(c) Why costly and how multilevel indices help:

- Binary search requires random I/O, which is expensive.
- **Solution: Multilevel index.**
 - Build an outer index on the inner index.
 - Example: For 10,000 blocks, outer index = 100 blocks ($\approx 400 \text{ KB}$).
 - Outer index fits in memory $\rightarrow$ only 1 random I/O needed.
- For very large files, more levels may be added $\rightarrow$ drastically reduce I/O.

Question

How are **point queries** and **range queries** processed using a B⁺-Tree index?

Answer

- **Point Query:**

1. Traverse from the root to a leaf using key comparisons.
2. If key $K_i = v$ exists at the leaf, return pointer P_i to the record.
3. If not found, return `null`.

- **Range Query:**

1. Traverse to the leaf where lb (lower bound) should be.
2. Scan that leaf and subsequent leaves using `next-leaf` pointers.
3. Collect all keys $C.K_i$ such that $lb \leq C.K_i \leq ub$.
4. Stop when a key $> ub$ is found or leaves end.

- **Efficiency:**

- Point queries need only $O(\log_{n/2} N)$ I/Os (3–4 disk reads for millions of records).
- Range queries benefit from sequential leaf linkage, minimizing random I/Os.

Question

What is the **cost of query processing** in a B⁺-Tree, and how does it compare with a binary search tree?

Answer

- **B⁺-Tree:**

- Height = $\lceil \log_{n/2}(N) \rceil$.
- With $n = 100$, $N = 1,000,000$: height ≈ 4 .
- Typically only 3–4 block reads (root often cached).

- **Binary Tree:**

- Height = $\lceil \log_2(N) \rceil$.
- With $N = 1,000,000$: height ≈ 20 .
- ≈ 20 random block reads per lookup.

- **Conclusion:** B⁺-trees are *fat and shallow*, binary trees are *tall and thin*. → B⁺-trees are much faster for disk-based queries.

Question

How do B⁺-Trees handle **non-unique keys** in an index?

Answer

- If attribute a_i is not unique (e.g., salary), duplicates may exist.
- To avoid duplicate keys in the index, build a **composite key** = (a_i, A_p) , where A_p is the primary key.
- Querying for all tuples with $a_i = v$:

`findRange((v, -∞), (v, +∞))`

returns all matching records.

- Range queries on a_i are handled in the same way.